Optimization and Machine Learning in VLSI Design Automation
Peking University, Fall 2026
Lecturer: Yibo Lin
Teaching Assistants: Shenglu Hua, Haichuan Liu
Release Date: September 28, 2026, 20:30
Due Date: October 13, 2026, 0:00

Homework 1: Logic Synthesis

Notation

All Boolean functions have values in $\{0, 1\}$. The Boolean derivative (Boolean difference) is

$$D_x f = f(x = 0, \mathbf{u}) \oplus f(x = 1, \mathbf{u}).$$

We use $\wedge$, $\vee$, and $\oplus$ for AND, OR, and XOR. All paths below are relative to the extracted homework directory.

Problem 1

Prove that

$$D_x(f \wedge g) = (f \wedge D_x g) \oplus (g \wedge D_x f) \oplus (D_x f \wedge D_x g). \tag{1}$$

Problem 2

Technology mapping transforms a technology-independent Boolean network into cells from a library.

(a) Prove that $\{\text{NAND2}, \text{NOT}\}$ is functionally complete: every Boolean function is realizable using only these gates. A NAND2 gate is $a \dagger b = \neg(a \wedge b)$; its two inputs may be connected to the same signal.

(b) Given

$$f(a, b, c, d) = (a \wedge d) \vee (b \wedge c \wedge d) \vee (a \wedge c),$$

rewrite f using only NAND2 and NOT.

(c) Let the technology library be $\{\text{NOT}, \text{NAND2}, \text{AND2}, \text{NOR2}, \text{AOI21}, \text{AOI22}\}$, where

$$\text{AOI21}(a, b, c) = \neg((a \wedge b) \vee c), \quad \text{AOI22}(a, b, c, d) = \neg((a \wedge b) \vee (c \wedge d)).$$

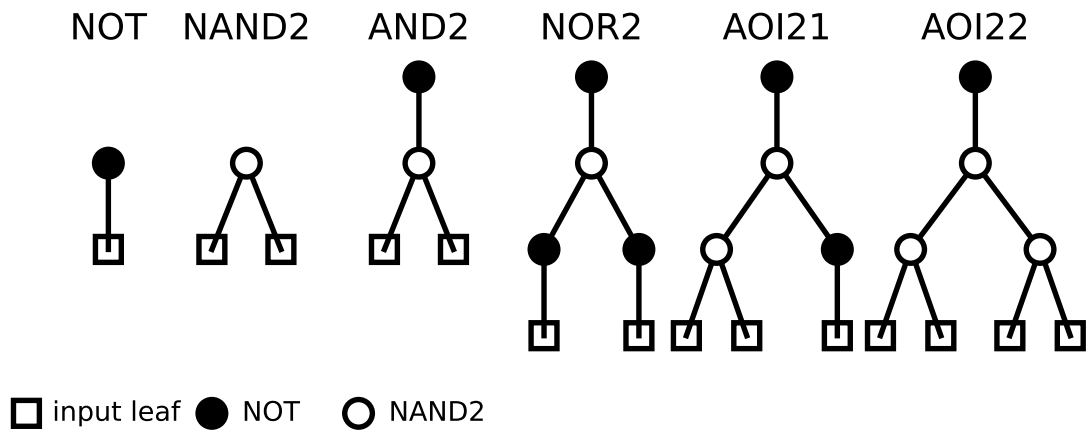


Figure 1: Each library gate drawn as a *pattern tree* over the base gates of part (b): an open node is NAND2, a filled node is NOT, and each square is an input leaf.

Figure 1 gives every library gate's pattern tree. A tree cover first writes the network in the same NAND2/NOT base form as in (b), then partitions it into pattern trees. Each library gate instance has unit cost, primary inputs are free, and internal subexpressions may not be shared. Find a minimum-cost tree cover of the function in (b); report the mapped circuit—which library gate realizes each part of the network and how the gates connect—and how many instances of each gate type it uses.

Problem 3

In Problem 2(c) you covered a small function by hand. Now automate the same covering on real designs, and run it inside the Lab-0 OpenROAD tool flow.

ABC. ABC is UC Berkeley's logic-synthesis and verification system: the `yosys-abc` binary at `$ORFS_ROOT/tools/install/yosys/bin/yosys-abc`, which `yosys synth` invokes for AIG-based optimization and mapping. It works internally on and-inverter graphs (AIGs) and ships a fast combinational equivalence checker (`cec`). In this problem it frames your work on both ends: it produced the subject graphs, and it verifies the netlist you emit. Your mapper fills the middle stage:

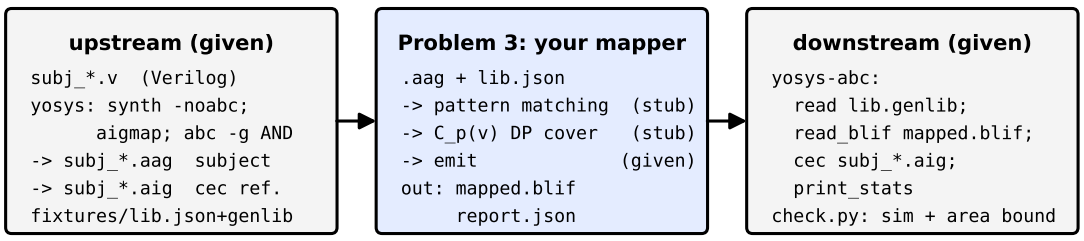


Figure 2: The Problem-3 pipeline. Upstream, the shipped fixtures are produced by Yosys and ABC (`abc -g AND` builds the subject AIG); downstream, ABC and the bundled `check.py` verify your output. Your part is the middle stage.

Each `subj_*` fixture is a real combinational block — among them an AES S-box and a 32-bit ALU assembled from the riscv32i datapath modules — synthesized into a subject graph (`subj_*.aag`: an AIG of 2-input AND nodes with optionally complemented edges, ASCII AIGER). The library is a 19-gate ASAP7 subset with real areas (`fixtures/lib.json`); `fixtures/README.md` is the format spec.

(a) The same covering as Problem 2(c), at netlist scale — with two twists: the subject is a *DAG*, and complemented edges matter. Write each literal as (v, p), the signal of node v for $p = 0$ and its complement for $p = 1$, and let $C_p(v)$ be the minimum area of a mapped subnetwork producing (v, p). A match m of gate g at v produces $(v, \text{out_inv}(g))$ and consumes covers of the distinct leaf literals $l_i = (v_i, p_i)$ it binds, so

$$C_p(v) = \min\Big\{ \min_{\substack{m \in \mathcal{M}(v) \\ \text{out_inv}(g_m) = p}} \text{area}(g_m) + \sum_i C_{p_i}(v_i),\ \text{area(inv1)} + C_{1-p}(v) \Big\},$$
$$C_0(\text{pi}) = 0, \qquad C_1(\text{pi}) = \text{area(inv1)},$$

where $\mathcal{M}(v)$ is the set of pattern-tree matches at v and `inv1` turns a net of one polarity into the other. In code this is a single bottom-up pass over the AIG:

```
for v in subject nodes, topological order:        # inputs first
    g0 = min{area(g) + sum C[l] : match of g at v, g.out_inv = 0}
    g1 = min{area(g) + sum C[l] : match of g at v, g.out_inv = 1}
    C[v][0] = min(g0, area(inv1) + g1)      # v  by inverting ~v's net
    C[v][1] = min(g1, area(inv1) + g0)      # ~v by inverting  v's net
```

Relate this to your hand computation in 2(c): (i) on a *tree* subject this recurrence is exactly the covering DP from 2(c) — argue why it is optimal there; (ii) on a DAG, is the sum of C_p over the primary-output literals still the emitted netlist's area? If not, what does the DP assume and where is optimality lost? (iii) 2(c)'s toy library hid a degree of freedom this one exposes: why does tracking both polarities beat paying one `inv1` per complemented edge?

(b) Implementation. The bundle ships `aig.py` (the `.aag` reader) and `map.py`, a skeleton that wires the pipeline end to end: the loaders and the emitter are given, so you fill two stubs whose contracts live in the file:

1. *Pattern matcher* — `matches_at(gate, var, subj)`: a recursive walk, exactly like the tree covering from the slides: align the pattern's root AND node with subject node v, try both child orders at each AND, require an internal pattern child's edge polarity to match the subject edge's exactly, and bind each leaf pin to the subject literal *before* the edge's internal inverter (a pin on several leaves must bind the same literal). The library already gives every gate's pattern tree in `lib.json`.
2. *Cover* — `cover(subj, gates)`: implement the bottom-up pass of part (a) over `subj.order`, whose file order is already topological. Store per literal its best cost and the winning choice, in the format `emit` expects; break ties in any consistent order, e.g. library order.

The shipped `emit` then expands the required output literals into `mapped.blif` and `report.json`. You may replace any piece with your own code against `fixtures/README.md`.

Hints. In the real fixtures, two details deserve attention:

- the subject is a DAG with several outputs, but no extra code is needed: your cost/choice tables are indexed by literal, so fanout and multi-output come for free — cover each output literal independently and let the emitter produce each required literal once. (Part (a) asks you to explain where the difference comes from.)
- patterns nest and repeat pins: `xor2` (Figure 3) puts pin `a` on two leaves under differently inverted edges. Each leaf binds the subject literal *before* its edge's internal inverter, and all leaves of one pin must agree.

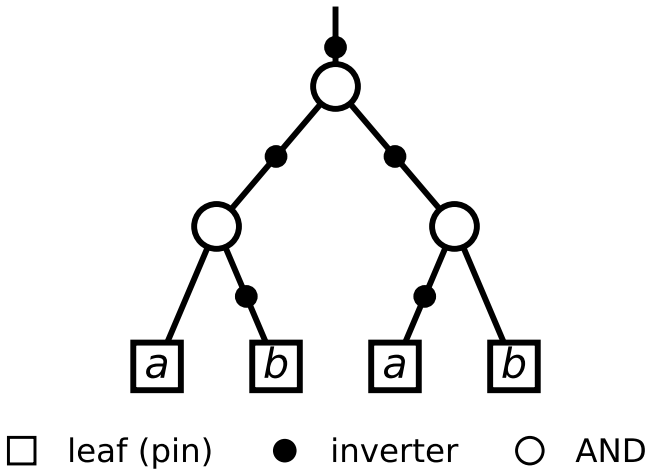


Figure 3: The `xor2` pattern tree: three AND nodes, each pin on two leaves.

(c) Deliverables. A single entry point that runs as

```
python3 map.py <subject.aag> <lib.json> -o mapped.blif -r report.json
```

and writes `mapped.blif` (one `.gate <gname> <pin>=<net> ... O=<net>` line per instance, with the subject's port names on `.inputs`/`.outputs`) and `report.json` (total area, cell count, per-gate counts).

(d) Acceptance. Run your mapper and the bundled checker on every subject (`p2func`, `alu4`, `sbox`, `riscv`):

```
for s in p2func alu4 sbox riscv; do
    python3 map.py fixtures/subj_$s.aag fixtures/lib.json \
        -o mapped_$s.blif -r report_$s.json
    python3 check.py --aag fixtures/subj_$s.aag --lib fixtures/lib.json \
        --golden fixtures/subj_${s}_golden.json \
        --net mapped_$s.blif --report report_$s.json
done
```

A subject counts as solved when the netlist is functionally correct on all golden vectors *and* its area stays within the bound in `fixtures/README.md` — reachable by the algorithm above; for a feel of the bar, a mapper that pays one `inv1` per complemented edge

overshoots every bound by 11–19 %. As a cross-check, ABC reads your netlist back (here for `alu4`):

```
yosys-abc -c "read fixtures/lib.genlib; read_blif mapped_alu4.blif; \
              cec fixtures/subj_alu4.aig; print_stats"
```

which should print the same area and report the networks equivalent; grading checks exact equivalence with `cec`.

Requirements

Submit a zip named `hw1-YOURSTUDENTID.zip` containing:

```
README.md
src/
hw1-YOURSTUDENTID.pdf
```

`src/` holds your implementation in any language. The README must state the exact command(s) that run your entry point on each subject.